

Reflection Journal Building and Optimizing a Chihuahua vs. Muffin Image Classifier

Summary of Workshop Objectives and Techniques

The main aim of this lab was to create, train, and evaluate an image classifier that would be able to differentiate between two tricky classes – Chihuahua and Blueberry Muffin. Performed in Google Colab environment, the lab covered the entire process of machine learning, from the creation of directory structure through the loading of data, creation of the neural network, its training, and evaluation. The main technologies used included the conversion of images to PyTorch tensors, normalization of pixels' values, and usage of simple MLPs and CNNs.

Key Concepts Learned

Image Classification: The basic machine learning process that involves building a model to analyze an image and give it the proper classification.

Differences between Fully Connected and Convolution Layers: Explaining why converting a 224×224 RGB image to a huge one-dimensional vector for a regular linear layer results in no spatial information at all, thus necessitating the use of CNNs to identify edges and textures.

Normalizing the Data and Tensors: Grasping the way that PyTorch deals with data loaders and how important normalization of pixel values is for stability in gradient descent.

Hyperparameter Tuning: Noticing how variations in learning rates, batch size, and epochs affect the model's accuracy.

Challenges Encountered and How They Were Overcome

The first hurdle presented itself right at the beginning of testing the baseline model (MySkynet), where it displayed a very low validation accuracy of 50/50, essentially making it the same as random guessing. The examination of the probability output demonstrated that the predictions were made in the range of 50% to 59%, which is due to the flatness of the network unable to recognize the difference between the wrinkles on the face of the dog and the muffin top. To solve this problem, we diagnosed the architectural problem as lack of spatial feature extraction and tried to find different solutions, one of which was changing the architecture.

Insights Gained About Machine Learning

In this project, it became evident that adding more dense layers to the raw pixel data was not a solution to the problem caused by a wrong choice of architecture. The images have spatial relationships between the pixels, meaning that they are useful; however, they lose their utility when converted to a one-dimensional vector. In addition, the project has proved that machine learning does not work as a "black box"; it is important to study the confidence scores and understand what happens during both forward and back propagation.

Potential Real-World Applications

Autonomous Robotics: Real-time detection of objects via vision is critical for self-driving cars and delivery robots to recognize people, obstructions, and traffic lights.

Security and Quality Control: Similar computer vision systems are used in industries and for security checks for automatic detection of structural flaws in the materials or categorization of scanned objects.

Medical Imaging: Convolutional neural networks are building blocks of the tools used by doctors for analysis of MRI and X-ray images to detect any abnormalities such as tumors or fractures.

Personal Reflections

My reflection on the workshop was that it was quite illuminating and cool since the process involved an in-depth exploration of the amount of work which goes on behind the scenes in computer vision. The process began by using the baseline MySkynet multilayer perceptron, which had failed in achieving any success at all due to the fact that the validation accuracy was only 50%. The process brought home the realization that it is important to have a thorough understanding of the theory and architectural designs when implementing neural networks. This realization made it clear that when dealing with neural networks, the approach taken should not be treating them as a black box and adding any kind of dense linear layers on raw image data. By doing so, the image becomes flattened into a 224×224 and in essence, the dog face and the muffin top becomes a mass of pixels and loses all of its structures. Troubleshooting through analyzing marginal confidence scores and adjusting training loop parameters until using convolutional neural network architecture enabled me to understand better. The understanding I gained was that convolutional and pooling layers can help in detecting

features such as edges, shapes, and textures. Moving forward, I'm super excited to take these concepts further, especially as I keep exploring robotics and automated systems where real-time visual perception is everything.

References

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems (NeurIPS 2019)*.